Ticker Modul Dokumentáció

Table of Contents

1. Általános . . . 1
   1.1. Architektúra . . . 2
   1.2. Használt technológiák . . . 2
   1.3. Projekt struktúra . . . 2
2. Projekt implementáció . . . 3
   2.1. Implementálás . . . 3
   2.2. Jobok konfigurációja . . . 4
   2.3. Helm config tudnivalók . . . 6
   2.4. Metrikák . . . 6
   2.5. Health . . . 7
   2.6. Version info . . . 8
   2.7. Job típusok . . . 8
   2.8. Microprofile Rest alapú job használata . . . 8
   2.9. Http hívás alapú job használata . . . 9
3. Hasznos parancsok és elérések . . . 10
   3.1. ticker-core-quartz-service Server indítása különböző módokon . . . 11
4. Projekt configurációs beállításainak lehetőségei . . . 11
   4.1. Quarkus alapú configok . . . 11
   4.2. Coffee alapú configok . . . 13
   4.3. Microprofile Openapi configok . . . 14
5. Release notes . . . 14
   5.1. ticker 1.3.0 . . . 14
      5.1.1. Változások . . . 14
      5.1.2. Migráció . . . 14

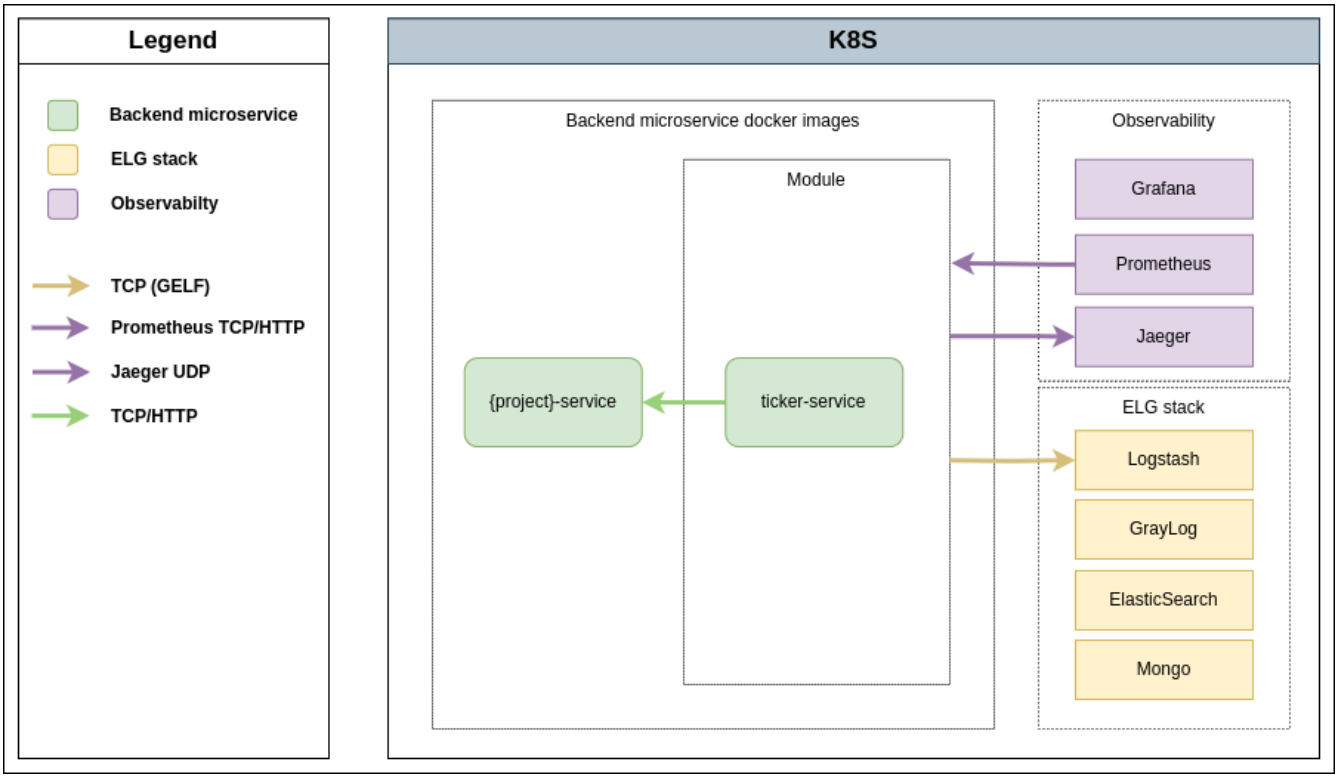
[Compile status] [Maven central version of Parent pom project] [License of Parent pom project] [Use JakartaEE project] [Commits] [Supported jvms] [GitHub Repo Stars]

1. Általános

A projekt célja, egy moduláris megoldás adása arra, hogy cron időzítéssel adott api végpontok meghívásával folyamatokat indítson el.

Dokumentáció: `https://i-cell-mobilsoft-open-source.github.io/ticker-backend/`

1.1. Architektúra



1.2. Használt technológiák

- Coffee 2.0.0+
  JavaEE solution set, melynek célja, hogy az enterprise világ gyakori algoritmusait összegyűjtse, azokra egy alapvető megoldást nyújtson, melyet, ha szükséges saját igényeinkre szabhatunk (https://github.com/i-Cell-Mobilsoft-Open-Source/coffee)
- Java 17+
- Maven 3.8.2+
- Jakarta EE 10
- CDI 4.0
- Microprofile 6.0
- Quarkus 3.2.5.Final

1.3. Projekt struktúra

- `/.github` - GitHub CI scriptek beállításai
- `/docs` - Dokumentációk tárhelye
- `/etc` - Konfigurációk, fejlesztési környezet scriptek, stb.
  - `/etc/config` - Konfigurációk
    - `/etc/config/grafana/**` - Grafana dashboardok, datasource-ok.
    - `/etc/config/META-INF/microprofile-config*.yml` - Microprofile által kezelt configurációs

yml fájlok
    - `/etc/config/prometheus/**` - Prometheus konfiguráció localhosthoz
    - `/etc/config/testsuite/META-INF/roaster-<profile>.yml` - Roaster alapú tesztekhez configurációs fájlok
    - `/etc/config/application.properties` - Quarkus alap configurációja
  - `/etc/docker-compose` - Docker compose fájlok és hozzájuk tartozó esetleges Dockerfile-ok.
  - `/etc/release` - Docker release kezelési seglet a GitHub actionben.
- `/ticker-bom` - Maven dependency definíciók
  - `/ticker-bom/ticker-bom-all` - Maven dependency management adó modul
  - `/ticker-bom/ticker-bom-project` - Projekt saját maven dependency definíciók
  - `/ticker-bom/ticker-bom-quarkus` - Quarkus maven dependency definíciók
- `/ticker-common` - Projekt általános elemeinek al moduljai
  - `/ticker-common-api` - Projekt api modulja
  - `/ticker-common-dto` - Projekt dto modulja
  - `/ticker-common-health` - Projekt health modulja
- `/ticker-core` - Projekt központi megvalósításainak almodulja
  - `/ticker-core-quartz` - Projekt központi magja, mely a ticker alapját adja quartz segítségével
- `/ticker-samples` - ticker samples al moduja
  - `/ticker-samples-mockapi` - Mock api végpontok microprofile rest clientjei
- `/ticker-services` - ticker service-el al moduja
  - `/ticker-core-quartz-service` - Mintaként használt quarkus alapú api ami bemutatja hogyan is lehet használni a ticker-core-quartz-ot saját local mp rest clientekkel.
- `/ticker-testsuite` - ticker sample service-hez tartozó testsuite a teszteléshez.

2. Projekt implementáció

2.1. Implementálás

A Ticker projekt két részre tagolható. Az egyik egy sampler és a hozzá tartozó testsuite alprojekt, a másik része pedig a ticker-core-quartz modul.

A sampler bemutatja, hogyan is kell használni a modult-t, hogy egy olyan servicet kapjunk ahol csak az **apit** kell dependency szinten összefűzni a **ticker-core-quartz modullal**.

Első lépés a ticker-core-quartz modul dependency-k közt való használata

```
<dependency>
    <groupId>hu.icellmobilsoft.ticker</groupId>
    <artifactId>ticker-core-quartz</artifactId>
```

```
</dependency>
```

Másodsorban azon api dependency használata, melyben az interface-ek vannak, amit a modul meg tud majd hívni:

Példa api dependency

```
<dependency>
    <groupId>hu.icellmobilsoft.sampler.api</groupId>
    <artifactId>api</artifactId>
</dependency>
```

Fontos, hogy mivel ez egy quarkus-on alapuló modul, így az implementáló modulnak is azonos verziójú quarkust használó alkalmazásnak kell lennie.

Ezek után, az alkalmazás indulásakor elindul a `SchedulerController`-ben definiált event.

Ekkor a `SchedulerController` látni fogja az indexelt interfaceket, és a megadott yml config szerint indítja őket.

2.2. Jobok konfigurációja

A futtatandó jobok a `application.yml` fájlban vannak konfigurálva.

Alap konfigurációk `microprofile-config.yml` szinten vannak konfigurálva a `ticker-core-quartz` dependencyben, ezért az implementáló projekten ezt a file-t ne írják felül

application.yml - Quarkus rest clientek megadása

```
quarkus:
    rest-client:
        ITickerTestRestRegisteredClient: ①
            url: http://localhost:8080 ②
            scope: jakarta.enterprise.context.ApplicationScoped ③
            read-timeout: 5000 ④
            connect-timeout: 5000 ⑤
```

Általános megoldása a Quarkusnak, hogy az alkalmazásban használandó rest clientek definíciói így adhatók meg.

A példában a legalapvetőbb configok vannak:

① A quarkus.rest-client config alatt a rest client interface azonosítóját kell megadni, ami maga az interface package-gel együtt, vagy ha `@RegisterRestClient` annotációban a configKey meg van adva, akkor az alapján kell hivatkozni itt rá.

② Az baseUri megadásának helye

③ az injectelt RestClient interface scope-jának meghatározása. Javasolt ApplicationScoped-ot használni, így sok problémát el lehet kerülni (rest client lezárás, memória veszteség stb.), figyelni kell arra, hogy ne próbáljuk a bean-t megsemmisíteni ha több job is használja az interfacet, mert hibához vezet. A thread-pool növelése ajánlott ha sok job kell ugyanazon a rest kliensen végig fusson.

④ Definiálni lehet válaszra várakozási timeoutot

⑤ Definiálni lehet kapcsolódási idő timeoutot

A 3-es pontban említett thead-pool növeléséhez a `DefaultRestClientBuilderListener` osztályt kell örökölni és felülírni az `onNewBuilder` metódusát amiben tudjuk állítani a thread-pool-t illetve a max-pooled-per-route paramétereket:

```
    @Override
    public void onNewBuilder(RestClientBuilder builder) {
        super.onNewBuilder(builder);
        builder.property("resteasy.connectionPoolSize",75);
        builder.property("resteasy.maxPooledPerRoute",50);
```

További lehetőségek miatt lásd: https://download.eclipse.org/microprofile/microprofile-rest-client-3.0/microprofile-rest-client-spec-3.0.html

application.yml - Jobok listájának definiálása

```
ticker:
    timer:
        activeJobs:
            - TEST_REST
            - TEST_REST_2
            - ..
```

A ticker.timer.activeJobs kulcs alatt listába sorolva lehet a jobok azonosító kulcsai alapján megadni, mely jobok fussanak.

Egy job definíciója

```
ticker:
    timer:
        ...
        job:
            TEST_REST: ①
                code: REST_QUARTZ_TEST ②
                cron: "*/10 * * ? * *" ③
                actionClass:
hu.icellmobilsoft.ticker.quartz.service.timer.job.mprestclient.MicroprofileRestClientJ
ob ④
                config: ⑤
                    mpRestClientClass:
```

```
hu.icellmobilsoft.ticker.sample.service.rest.test.api.ITickerTestRestRegisteredClient
                    method:
getTest(java.lang.String,java.lang.Integer,java.lang.Long,java.lang.Boolean,java.time.
OffsetDateTime)
                    parameters:
                        - config
                        - 50
                        - 30
                        - true
                        - 2023-05-24T19:10:24.136+05:30
```

① - Job azonosítója, mely a fenti listában van hivatkozva.

② - A code segítségével lehet a logokban keresni az adott jobot. Healt-ben is <code>-Job néven található meg, és a logokba is pl.: `<<< End quartz job type [REST_QUARTZ_TEST job]`

③ - Cron beállítás

④ Action osztály, ami definiálja a job folyamatát. A jelenlegi példában a `MicroprofileRestClientJob` van használva, ami egy MP rest clientet tud meghívni.

⑤ Configurációs paraméterei a 4. pontban használt jobnak. A jelenlegi példában az mpRestClientClass -t kell megadni, metódust hogy melyiket szeretnénk a jobban hívni és a paramétereket.

2.3. Helm config tudnivalók

A quarkus támogatja a dev és test profilokat, és engedélye más profilok létrehozását is, viszont minden környezetben más és más értékei lehetnek egy-egy konfigurációnak, mint pl egy url.

Emiatt Helm config szintjén kell beállítani egyedi microprofile-config.yml fájlt, ami felülírja az alkalmazásban lévő configurációkat.

A quarkus ad lehetőséget környezeti változón keresztüli config source beállításra: https://quarkus.io/guides/config-reference#quarkus-config-config_quarkus.config.locations

Tehát helm values esetén a következőket kell beállítani:

values.yaml

```
configMountPath: /deployments/app/config
...
extraEnv:
- name: QUARKUS_CONFIG_LOCATIONS
value: {{ .Values.configMountPath }}/microprofile-config.yml
```

2.4. Metrikák

A `hu.icellmobilsoft.ticker.quartz.service.quartz.util.QuartzJobUtil` osztály szolgáltatja a Quartz Job-okról a metrikákat. Az `org.quartz.Scheduler`-be az `org.quartz.ListenerManager` interface-en keresztül adjuk hozzá a saját

`hu.icellmobilsoft.ticker.quartz.service.quartz.health.metric.MetricJobListener`-ünket ami az `org.quartz.JobListener`-t implementálja.

Jelenleg az következő Quartz Job metrikák érhetőek el:

- Quartz job prev fire time
  - Az előző Job futás időpontja
  - kulcs: `quartz_job_prev_fire_time`
- Quartz job next fire time
  - A következő Job futás időpontja
  - kulcs: `quartz_job_next_fire_time`
- Quartz job run time
  - A legutóbbi Job futás ideje
  - kulcs: `quartz_job_run_time`

A metrikákat az alkalmazásszerver a `<host:port>/q/metrics` végponton szolgáltatja `application_` prefix-szel ellátva.

példa

```
application_quartz_job_prev_fire_time{configKey="REST_QUARTZ_TEST-Job",quantile="0.5"}
1.66921282E12
application_quartz_job_next_fire_time{configKey="REST_QUARTZ_TEST-Job",quantile="0.5"}
1.66921283E12
application_quartz_job_run_time{configKey="REST_QUARTZ_TEST-Job",quantile="0.5"} 41.0
```

2.5. Health

A **MicroProfile Health** specifikációt implementáló **SmallRye Health** projekten keresztül van megvalósítva. A metrikákat az alkalmazásszerver a `<host:port>/q/health` végponton szolgáltatja.

példa válasz

```
{
    "status": "UP",
    "checks": [
        {
            "name": "ticker-quartz",
            "status": "UP",
            "data": {
                "quarkusUUID": "97debf17-5e9f-41ab-982a-c4bffc895765",
                "REST_QUARTZ_TEST-Job": "PreviousFireTime [2022-12-
07T12:38:30.000+0000], NextFireTime [2022-12-07T12:38:40.000+0000]",
                "REST_QUARTZ_TEST_2-Job": "PreviousFireTime [2022-12-
07T12:38:30.000+0000], NextFireTime [2022-12-07T12:38:35.000+0000]"
            }
```

```
        }
    ]
}
```

Elérhető egy kísérleti **health-ui**. Alapértelmezetten dev és test módban aktív, de production módban is be lehet konfigurálni (viszont ez csak build time property, nem lehet runtime állítani: `quarkus.smallrye-health.ui.always-include=true`) és a `<host:port>/q/health-ui` url-en érhető el böngészőben.

2.6. Version info

Elérhető `<host:port>/versionInfo` végpont, amely visszadja a jelenlegi implementáció verziót.

példa válasz

```
Manifest-Version: 1.0
Class-Path:
Main-Class: io.quarkus.runner.GeneratedMain
Implementation-Title: ticker-core-quartz-service
Implementation-Version: 0.3.0-SNAPSHOT
Multi-Release: true
```

2.7. Job típusok

A servicebe több job típus használatára van lehetőség

2.8. Microprofile Rest alapú job használata

A job lehetővé teszi http hívások kezdeményezését mp rest client segítségével.

A következő példa bemutatja a használatot:

application.yml

```
ticker:
    timer:
        activeJobs:
            - TEST_REST ①
        job:
            TEST_REST: ②
                code: REST_QUARTZ_TEST ③
                cron: "*/10 * * ? * *" ④
                actionClass:
hu.icellmobilsoft.ticker.quartz.service.timer.job.mprestclient.MicroprofileRestClientJ
ob ⑤
                config:
                    mpRestClientClass:
```

```
hu.icellmobilsoft.ticker.sample.service.rest.test.api.ITickerTestRestRegisteredClient
⑥
                    method:
getTest(java.lang.String,java.lang.Integer,java.lang.Long,java.lang.Boolean,java.time.
OffsetDateTime) ⑦
                    parameters: ⑧
                        - config
                        - 50
                        - 30
                        - true
                        - 2023-06-07T13:45:27.893013372Z
```

① - a ticker.timer.activeJobs alatt lehet definiálni mely job legyen aktív

② - a ticker.time.job alatt definiálhatók a jobok, amelyekre az első pontban van hivatkozás

③ - A code segítségével lehet a logokban keresni az adott jobot. Health-ben is <code>-Job néven található meg, és a logokba is pl.: `<<< End quartz job type [REST_QUARTZ_TEST job]`

④ - Cron beállítás

⑤ - Action osztály, ami definiálja a job folyamatát.

⑥ - Az action configja, ahol a mpRestClientClass adható meg ami a rest client interface adható meg.

⑦ - Az action configja, ahol a method adható meg a rest client interface-en belül

⑧ - Az action configja, ahol a parameterek adható meg a metódus híváshoz. Bármelyik lista elem definiálható static metódus hívás is definiálható a { kezdettel és } befejezéssel.

2.9. Http hívás alapú job használata

A job lehetővé teszi a legalapvetőbb http hívások definiálását. Az egyetlen megoldanó feladat ha egyedi body kialakítás szükséges például egy POST híváshoz, amit az eddigi metódus definiálással megoldható.

A következő példa bemutatja a használatot:

application.yml

```
ticker:
    timer:
        activeJobs:
            - TEST_APACHE_HTTP_CLIENT ①
        job:
            TEST_APACHE_HTTP_CLIENT: ②
                code: TEST_APACHE_HTTP_CLIENT ③
                cron: "*/1 * * ? * *" ④
                actionClass:
hu.icellmobilsoft.ticker.quartz.service.timer.job.httpclient.HttpClientJob ⑤
                config:
                    baseUrl: http://localhost:8080/test/ticker ⑥
                    method: Get ⑦
                    body:
```

```
"&{hu.icellmobilsoft.ticker.common.util.version.BaseRequestUtil.generate}" # static
method call ⑧
                    headers:
                        Content-Type: "application/xml"
                        Accept: "application/json"
                    queryParams:
                       testString: value
                       testInteger: 1000
                       testLong: 50000
                       testBoolean: true
                       testOffsetDateTime: 2023-06-07T13:45:27.893013372Z
```

① - a ticker.timer.activeJobs alatt lehet definiálni mely job legyen aktív

② - a ticker.time.job alatt definiálhatók a jobok, amelyekre az első pontban van hivatkozás

③ - A code segítségével lehet a logokban keresni az adott jobot. Healt-ben is <code>-Job néven található meg, és a logokba is pl.: `<<< End quartz job type [TEST_APACHE_HTTP_CLIENT job]`

④ - Cron beállítás

⑤ - Action osztály, ami definiálja a job folyamatát, a példában lehet a Http hívás alapú jobot használni.

⑥ - Az action configja, ahol a baseUrl adható meg a http híváshoz

⑦ - Az action configja, ahol a method adható meg a http híváshoz

⑧ - Az action configja, ahol a body adható meg a http híváshoz. A body-ba static metódus hívás is definiálható a &{ kezdettel és } befejezéssel.

⑨ - Az action configja, ahol a headerök adhatók meg a http híváshoz

⑩ - Az action configja, ahol a queryParams adható meg a http híváshoz

3. Hasznos parancsok és elérések

Fejlesztés céljából használt parancsok, melyek a fejlesztői környezetek felépítésére és indítására szolgálnak.

Az alkalmazás több féle módon indítható:

- Quarkus dev indítása mavennel
- Quarkus uber-jar készítése és ezen jar fájl futtatása `java -jar` segítségével
  - Ugyan ez a jar java docker image-be téve és megfuttatva (Dockerfile.uber-jar.jvm használata)

Docker image-ek készítésére és futtatására docker-compose van használva.

> A projekt tartalmaz egy sampler service-t ami megmutatja a modul használatát. Ez a példa képes teljes egészben csak lokális fejlesztői gépen futni. Tehát nincs semmi külső függősége.

3.1. ticker-core-quartz-service Server indítása különböző módokon

IDE included Quarkus run config

```
Több böngésző is nyújt támogatást natívan a quarkussal kapcsolatoban, úgy mint a
spring boot projektekhez, hogy felismeri és saját futtatási konfigurációt hoz létre.
```

Maven quarkus:dev

```
mvn clean compile quarkus:dev
```

A projekt nem 1 modulból áll, mint ahogy a quarkus elvárná, emiatt kell a compile.

A quarkus maven plugin segítségével dev módban indítható a projekt, így több dev tool is aktiválódik. További információ: https://quarkus.io/guides/dev-mode-differences.

Quarkus uber-jar futtatása dockerben

```
mvn clean install ①
docker-compose -f <PROJECT_PATH>/ticker-backend/etc/docker-compose/docker-
compose.local.ticker-service.yml up --build --force-recreate ②
```

① azért szükséges hogy a jar ami a docker image-be kerül előálljon.

② docker compose parancs a projekt root-jában kiadva indítja a docker-compose buildet (force recreate paraméterrel kierőszakolva az image újrabuildelését, ha lenne), és a felfutást (up)

4. Projekt configurációs beállításainak lehetőségei

4.1. Quarkus alapú configok

Az alkalmazás mivel Quarkus alapú, így a quarkus alap beállításai használhatóak benne.

A leírás itt található meg: https://quarkus.io/version/3.2/guides/all-config

A configuráció listából, csak azon elemek aktívak, melyek dependency szinten be vannak téve a projektbe.

Fontosabb elemek, melyek alapból már definiálva vannak a projekttel:

Quarkus config key	Description	Env variable	Default value
quarkus.arc.remove-unused-beans	Arc setting - remove unused beans: Link	-	false
quarkus.log.category."hu.icellmobilsoft".level	hu.icellmobilsoft category log level	TICKER_LOG_HU_ICELLMOBILSOFT_LEVEL	INFO
quarkus.log.console.json	Json logging enable	TICKER_LOG_CONSOLE_JSON_ENABLED	false
quarkus.log.console.format	Console log format	-	`%d{yyyy-MM-dd HH:mm:ss.SSS} %-5p [thread:%t] [%c{10}] [sid:%X{extSessionId}] - %s%E%n`
quarkus.log.handler.gelf.additional-field."moduleVersion".value	Gelf log - **moduleVersion** additional-field value	TICKER_LOGSTASH_MODULE_VERSION	unknown
quarkus.log.handler.gelf.additional-field."moduleId".value	Gelf log - **moduleId** additional-field value	TICKER_LOGSTASH_MODULE_ID	unknown
quarkus.log.handler.gelf.additional-field."K8S_NAMESPACE".value	Gelf log - **K8S_NAMESPACE** additional-field value	TICKER_LOGSTASH_K8S_NAMESPACE	unknown
quarkus.handler.gelf.include-full-mdc	Gelf log - Whether to include all fields from the MDC.	TICKER_LOGSTASH_K8S_NAMESPACE	false
quarkus.log.level	Quarkus log level: Link	TICKER_LOG_LEVEL	INFO
quarkus.log.min-level	Quarkus min log level: Link	TICKER_LOG_MIN_LEVEL	ALL
quarkus.jaeger.enabled	Jaeger - enabled config: Link	TICKER_JAEGER_ENABLED	false
quarkus.jaeger.endpoint	Jaeger - : https://[Link]	QUARKUS_JAEGER_ENDPOINT=http://jaeger:14268/api/traces	• (for example: http://jaeger:14268/api/traces)
quarkus.package.add-runner-suffix	Quarkus package add runner suffix: Link	-	false
quarkus.package.type	Quarkus package type: Link	-	uber-jar
quarkus.quartz.clustered	Quartz - clustered : Link	-	false
quarkus.quartz.thread-count	Quartz - thread count : Link	TICKER_QUARTZ_THREAD_COUNT	25

Quarkus config key	Description	Env variable	Default value
quarkus.scheduler.start-mode	Quartz - start mode : Link	-	FORCED
quarkus.smallrye-openapi.info-title	Openapi - info title : Link	-	Ticker service
quarkus.smallrye-openapi.info-version	Quartz - info version : Link	-	${quarkus.application.version}
quarkus.smallrye-openapi.info-description	Quartz - info version : Link	-	REST endpoints for operations. General responses in case of error: * __400__ - Bad Request * __401__ - Unauthorized * __404__ - Not found * __418__ - Database object not found * __500__ - Internal Server Error
quarkus.swagger-ui.enable	Enable swagger ui: Link	-	false

4.2. Coffee alapú configok

Az alkalmazás ezen felül a coffee toolset használata miatt, további konfigurációs lehetőségeket tartalmaz.

Coffee config key	Description	Env variable	Default value
coffee.app.name	Coffee app name in logs	-	${quarkus.application.name}
coffee.config.resource.bundles	Resource bundles' config for i18n	-	i18n.common-messages,i18n.messages
coffee.config.xml.catalog.path	Catalog path of Super catalog.xml	-	xsd/hu/icellmobilsoft/ticker/dto/super.catalog.xml

4.3. Microprofile Openapi configok

Ezen kívül coffeehoz kapcsolódóan, tartalmaz egy microprofile openapi filter configurációt is

MP Openapi config key	Description	Env variable	Default value
mp.openapi.filter	Microprofile openapi filter class with package	-	hu.icellmobilsoft.ticker.common.rest.filter.OpenAPIFilter

5. Release notes

5.1. ticker 1.3.0

5.1.1. Változások

- opensource projekt létesítése

5.1.2. Migráció

A változások visszafelé kompatibilisek nem igényel migrációt.